

Practice 2 실행 보고서

1번

[SQL]

```
1  -- 문제 1
2  -- 학생과 수강을 INNER JOIN하기
3  -- 수강 존재 학생의 과목/성적을 조회
4  SELECT
5      s.name as 학생,
6      e.course as 과목,
7      e.grade as 성적
8  FROM student as s
9  INNER JOIN enroll as e
10     ON e.student_id = s.student_id
11  order by s.student_id asc, e.course asc
12  limit 5;
```

[실행결과]

```
heejoo@munhuijuui-MacBookPro scala-postgre
ice2/problem1.sql
학생 | 과목 | 성적
-----|-----|-----
Student_1 | Course_3 | C
Student_1 | Course_4 | D
Student_1 | Course_5 | A
Student_2 | Course_4 | D
Student_2 | Course_5 | A
(5 rows)
```

2번

[SQL]

```
sql > practice2 > problem2.sql
1  -- 문제 2
2  -- 모든 학생 기준으로 수강을 붙이고,
3  -- 과목(없으면 NULL)까지 보이기
4  SELECT
5      s.name as 학생,
6      e.course as 과목
7  FROM student as s
8  LEFT JOIN enroll as e
9      ON s.student_id = e.student_id
10  ORDER BY s.student_id asc, e.course asc
11  limit 5;
```

[실행결과]

```
heejoo@munhuijuui-MacBookPro scala-post
ice2/problem2.sql
  학생 | 과목
-----+-----
Student_1 | Course_3
Student_1 | Course_4
Student_1 | Course_5
Student_2 | Course_4
Student_2 | Course_5
(5 rows)
```

3번

[SQL]

```
1  -- 문제 3
2  -- 수강이 기준. 학생이 없으면 학생 정보가 NULL
3  SELECT
4  |   e.course as 과목,
5  |   s.*
6  FROM student as s
7  RIGHT JOIN enroll as e
8  |   ON s.student_id = e.student_id
9  ORDER BY e.course asc, s.student_id asc
10 limit 5;
```

[실행결과]

```
ice2/problem3.sql
  과목 | student_id | name | major | gpa
-----+-----+-----+-----+-----
AI
Course_1 | 17 | Student_17 | ME | 3.70
Course_1 | 18 | Student_18 | CE | 3.80
Course_1 | 19 | Student_19 | BIO | 3.90
Course_1 | 37 | Student_37 | ME | 2.70
(5 rows)
```

4번

[SQL]

```

sql > practice2 > problem4.sql
1  -- 문제 4
2  -- 학생/수강 모두 포함
3  SELECT
4  |   *
5  FROM student as s
6  FULL OUTER JOIN enroll as e
7  |   ON e.student_id = s.student_id
8  ORDER BY e.course asc, s.student_id asc
9  limit 5;

```

[실행결과]

```
ice2/problem4.sql
```

student_id	name	major	gpa	student_id	course	grade
				1001	AI	A
17	Student_17	ME	3.70	17	Course_1	A
18	Student_18	CE	3.80	18	Course_1	A
19	Student_19	BIO	3.90	19	Course_1	A
37	Student_37	ME	2.70	37	Course_1	A

(5 rows)

5번

[SQL]

```

sql > practice2 > problem5.sql
1  -- 문제 5
2  -- 한 번도 수강하지 않은 학생 목록
3  SELECT
4  |   s.*
5  FROM student as s
6  WHERE NOT EXISTS (
7  |   SELECT e.student_id
8  |   FROM enroll as e
9  |   where s.student_id = e.student_id
10 | )
11 ORDER BY s.student_id ASC
12 LIMIT 5;

```

[실행결과]

```
ice2/problem5.sql
```

student_id	name	major	gpa
10	Student_10	CS	3.00
20	Student_20	CS	4.00
30	Student_30	CS	2.00
40	Student_40	CS	3.00
50	Student_50	CS	4.00

(5 rows)

6번

[SQL]

```
sql > practice2 > problem6.sql
1  -- 문제 6
2  -- 한 과목 이상 수강한 학생 목록(중복 제거)
3  SELECT
4      s.*
5  FROM student s
6  WHERE s.student_id IN (
7      SELECT
8          DISTINCT e.student_id
9      FROM enroll e
10     where e.student_id = s.student_id
11 )
12 ORDER BY s.student_id ASC
13 limit 5;
```

[실행결과]

```
ice2/problem6.sql
student_id | name      | major | gpa
-----|-----|-----|-----
1 | Student_1 | EE     | 2.10
2 | Student_2 | ME     | 2.20
3 | Student_3 | CE     | 2.30
4 | Student_4 | BIO    | 2.40
5 | Student_5 | CS     | 2.50
(5 rows)
```

7번

[SQL]

```
sql > practice2 > problem7.sql
1  -- 문제 7
2  -- 고객별 주문건수/총액
3  SELECT
4      c.customer_name as 이름,
5      count(order_id) as 주문건수,
6      sum(o.amount) as 총액
7  FROM customers c
8  INNER JOIN orders o
9      ON c.customer_id = o.customer_id
10 GROUP BY c.customer_id
11 ORDER BY c.customer_id asc
12 limit 5;
```

[실행결과]

```
ice2/problem7.sql
이름 | 주문 건 수 | 총 액
-----|-----|-----
Customer_1 | 6 | 16530.00
Customer_2 | 6 | 12608.06
Customer_3 | 6 | 12686.12
Customer_4 | 6 | 12764.18
Customer_5 | 6 | 12842.24
(5 rows)
```

8번

[SQL]

```
sql > practice2 > problem8.sql
1  -- 문제 8
2  -- 총액 상위 10명과 금액
3  SELECT
4      c.customer_id as 고객번호,
5      c.customer_name as 고객이름,
6      sum(o.amount) as 총액
7  FROM customers c
8  INNER JOIN orders o
9      ON c.customer_id = o.customer_id
10 GROUP BY c.customer_id
11 ORDER BY 총액 desc, c.customer_id asc
12 limit 10;
```

[실행결과]

```
ice2/problem8.sql
고객 번호 | 고객 이름 | 총 액
-----|-----|-----
154 | Customer_154 | 19467.18
153 | Customer_153 | 19389.12
152 | Customer_152 | 19311.06
151 | Customer_151 | 19233.00
150 | Customer_150 | 19154.94
149 | Customer_149 | 19076.88
148 | Customer_148 | 18998.82
147 | Customer_147 | 18920.76
146 | Customer_146 | 18842.70
145 | Customer_145 | 18764.64
(10 rows)
```

9번

[SQL]

```

sql > practice2 > problem9.sql
1  -- 문제 9
2  -- 모든 직원과 그 매니저 이름
3  -- 별칭에 공백 존재 시 큰 따옴표로 감싸야 함
4  SELECT
5      e.emp_id as "직원 번호",
6      e.name as "직원 이름",
7      m.name as "담당 매니저 이름"
8  FROM emp e
9  LEFT JOIN emp m
10     ON e.manager_id = m.emp_id
11  ORDER BY e.emp_id asc;

```

[실행결과]

```

ice2/problem9.sql
1 | CEO |
2 | Mgr_2 | CEO
3 | Mgr_3 | CEO
4 | Mgr_4 | CEO
5 | Mgr_5 | CEO
6 | Mgr_6 | CEO
7 | Mgr_7 | CEO
8 | Mgr_8 | CEO
9 | Mgr_9 | CEO
10 | Mgr_10 | CEO
11 | Mgr_11 | CEO
12 | Dev_12 | CEO
13 | Dev_13 | Mgr_2
14 | Dev_14 | Mgr_3
15 | Dev_15 | Mgr_4
16 | Dev_16 | Mgr_5
17 | Dev_17 | Mgr_6
18 | Dev_18 | Mgr_7
19 | Dev_19 | Mgr_8
20 | Dev_20 | Mgr_9
21 | Dev_21 | Mgr_10
22 | Dev_22 | CEO
23 | Dev_23 | Mgr_2
24 | Dev_24 | Mgr_3
25 | Dev_25 | Mgr_4

```

10번

[SQL]

```

sql > practice2 > problem10.sql
1  -- 문제 10
2  -- "모든 학생 기준"으로 과목 분포를 보고 싶다
3  -- → LEFT JOIN + 집계
4  SELECT
5      COALESCE(e.course, '미수강') as "과목",
6      COUNT(DISTINCT s.student_id) as "수강 인원"
7  FROM student s
8  LEFT JOIN enroll e
9      ON e.student_id = s.student_id
10 GROUP BY e.course
11 ORDER BY e.course desc, e.course asc
12 limit 5;

```

[실행결과]

```
ice2/problem10.sql
```

과목	수강 인원
미수강	100
DB	107
Course_9	144
Course_8	94
Course_7	141

(5 rows)

11번

[SQL]

```

sql > practice2 > problem11.sql
Changed lines 문제 11
2  -- DB 과목을 듣지 않은 모든 학생을 나열
3  -- LIMIT을 쓰면 반환 순서가 달라짐
4  SELECT
5      s.student_id as "학번",
6      s.name as "학생 이름"
7  FROM student s
8  WHERE NOT EXISTS (
9      SELECT 1
10     FROM enroll e
11     WHERE e.student_id = s.student_id
12           AND e.course = 'DB'
13 )

```

[실행결과]

LIMIT 여부에 따라 순서가 달라짐

PostgreSQL은 LIMIT N이 붙어 있으면 "전부 다 찾을 필요 없이 상위 N개만 빠르게 찾으면 끝내자"는 방향으로 실행 계획을 전면적으로 바꾸기 때문에 발생하는 현상

```
heejoo@munhuijuui-MacBookPro
ice2/problem11.sql
106 | Student_106
681 | Student_681
285 | Student_285
120 | Student_120
866 | Student_866
887 | Student_887
```

```
ice2/problem11.sql
학 번 | 학생 이름
-----+-----
1 | Student_1
2 | Student_2
3 | Student_3
4 | Student_4
5 | Student_5
(5 rows)
```

12번

[SQL]


```

sql > practice2 > problem12.sql
1  -- 문제 12
2  -- (가정) 과목별로 매니저가 운영 책임을 갖는다고 가정하고,
3  -- emp의 매니저(이름 'Mgr_'로 시작)와 과목을 임의로 매핑한 테이블
4  -- course_owner(course, manager_id)를 만든 뒤,
5  -- 과목별 수강 인원 + 책임 매니저 이름 리포트를 작성하세요
6
7  -- course_owner 테이블 생성
8  -- 랜덤 매핑하려다가 확인용 데이터만 넣었음
9  CREATE TABLE IF NOT EXISTS course_owner (
10     course VARCHAR(50) PRIMARY KEY,
11     manager_id INT NOT NULL REFERENCES emp(emp_id)
12 );
13 INSERT INTO course_owner (course, manager_id)
14     VALUES
15     ('DB', 2),
16     ('AI', 3),
17     ('ML', 4),
18     ('Course_1', 5),
19     ('Course_2', 6),
20     ('Course_3', 7),
21     ('Course_4', 8),
22     ('Course_5', 9),
23     ('Course_6', 10),
24     ('Course_7', 11)
25     ON CONFLICT (course) DO NOTHING; -- 이미 같은 과목이 있으면 skip
26
27 -- 과목별 수강 인원 + 책임 매니저 이름
28 SELECT
29     e.course as "과목 이름",
30     COUNT(DISTINCT e.student_id) as "수강 인원",
31     m.name as "책임 매니저 이름"
32 FROM enroll e
33 LEFT JOIN course_owner co
34     ON co.course = e.course
35 JOIN emp m
36     ON m.emp_id = co.manager_id
37 GROUP BY e.course, m.name
38 ORDER BY e.course;

```

[실행결과]

```
ice2/problem12.sql
psql:sql/practice2/problem12.sql:12: NOTICE
ipping
CREATE TABLE
INSERT 0 0

```

과목 이름	수강 인원	책임 매니저 이름
AI	1	Mgr_3
Course_1	144	Mgr_5
Course_2	47	Mgr_6
Course_3	94	Mgr_7
Course_4	94	Mgr_8
Course_5	141	Mgr_9
Course_6	94	Mgr_10
Course_7	141	Mgr_11
DB	107	Mgr_2
ML	1	Mgr_4

```
(10 rows)
```

13번

[SQL]

```
sql > practice2 > problem13.sql
1  -- 문제 13
2  -- 학생 x 과목 전체 조합을 만들어 "학생별 과목 추천 후보"를 만들되,
3  -- 샘플 100건만 본다
4  SELECT
5      s.name as "학생 이름",
6      e.course as "추천 과목"
7  FROM student s
8  CROSS JOIN enroll e
9  LIMIT 100;
```

[실행결과]

```
ice2/problem13.sql
Student_1 | Course_3
Student_2 | Course_3
Student_3 | Course_3
Student_4 | Course_3
Student_5 | Course_3
Student_6 | Course_3
Student_7 | Course_3
Student_8 | Course_3
Student_9 | Course_3
Student_10 | Course_3
```

14번

[SQL]

```

sql > practice2 > problem14.sql
1  -- 문제 14
2  -- 스칼라 서브쿼리 (SELECT 절) 사용
3  -- 학생 + 소속 학과명 붙이기
4  SELECT
5      s.student_id as "학번",
6      s.name || (SELECT s.major) as "학생이름 + 소속 학과"
7  FROM student s
8  ORDER BY s.student_id
9  LIMIT 5;|

```

[실행결과]

```

ice2/problem14.sql
학번 | 학생이름 + 소속 학과
-----|-----
1 | Student_1EE
2 | Student_2ME
3 | Student_3CE
4 | Student_4BIO
5 | Student_5CS
(5 rows)

```

15번

[SQL]

```

sql > practice2 > problem15.sql
1  -- 문제 15
2  -- 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)
3  -- GPA가 NULL인 경우 제외 ... NULL 데이터 없음
4  SELECT
5      s.student_id as "학번",
6      s.name as "이름",
7      s.major as "전공",
8      s.gpa as "GPA"
9  FROM student s
10 WHERE s.gpa > (
11     SELECT AVG(gpa)
12     FROM student
13 )
14 ORDER BY student_id
15 LIMIT 5;|

```

[실행결과]

```

15.sql
학번 | 이름 | 전공 | GPA
-----|-----|-----|-----
15 | Student_15 | CS | 3.50
16 | Student_16 | EE | 3.60
17 | Student_17 | ME | 3.70
18 | Student_18 | CE | 3.80
19 | Student_19 | BIO | 3.90
(5 rows)

```

16번

[SQL]

```
sql > practice2 > problem16.sql
1  -- 문제 16
2  -- 자신의 학과 평균 GPA보다 높은 학생
3  -- (Correlated subquery)
4  SELECT
5      s.student_id as "학번",
6      s.name as "이름",
7      s.major as "전공",
8      s.gpa as "GPA"
9  FROM student s
10 WHERE s.gpa > (
11     SELECT AVG(tmp.gpa)
12     FROM student tmp
13     WHERE tmp.major = s.major
14 )
15 ORDER BY s.student_id
16 LIMIT 5;
```

[실행결과]

ice2/problem16.sql			
학번	이름	전공	GPA
15	Student_15	CS	3.50
16	Student_16	EE	3.60
17	Student_17	ME	3.70
18	Student_18	CE	3.80
19	Student_19	BIO	3.90
(5 rows)			

17번

[SQL]

```

sql > practice2 > problem17.sql
1  -- 문제 17
2  -- 수강(enroll) 기록이 있는 학생만
3  -- 수강 기록이 있는 학생들의 평균 GPA보다 높은 GPA를 가진 학생 목록
4  with tmp as (
5      SELECT
6          s.student_id,
7          s.name,
8          s.major,
9          s.gpa
10     FROM student s
11     WHERE s.student_id IN (
12         SELECT DISTINCT e.student_id
13         FROM enroll e
14     )
15 )
16 SELECT
17     s.student_id as "학번",
18     s.name as "이름",
19     s.major as "전공",
20     s.gpa as "GPA"
21 FROM student s
22 WHERE s.gpa > (
23     SELECT AVG(gpa)
24     FROM tmp
25 )
26 ORDER BY s.student_id
27 LIMIT 5;

```

[실행결과]

```
ice2/problem17.sql
```

학 번	이름	전 공	GPA
15	Student_15	CS	3.50
16	Student_16	EE	3.60
17	Student_17	ME	3.70
18	Student_18	CE	3.80
19	Student_19	BIO	3.90

(5 rows)

18번

[SQL]

```

sql ~/workspace/skala-postgre/sql/practice2/problem19.sql • Modified
1  -- 문제 18
2  -- 한 번도 수강하지 않은 학생
3  -- 수강 기록이 없는 학생들의 평균 GPA보다 높은 GPA를 가진 학생 목록
4  with tmp as (
5      SELECT
6          s.student_id,
7          s.name,
8          s.major,
9          s.gpa
10     FROM student s
11     WHERE NOT EXISTS (
12         SELECT 1
13         FROM enroll e
14         WHERE e.student_id = s.student_id
15     )
16 )
17 SELECT
18     s.student_id as "학번",
19     s.name as "이름",
20     s.major as "전공",
21     s.gpa as "GPA"
22 FROM student s
23 WHERE s.gpa > (
24     SELECT AVG(gpa)
25     FROM tmp
26 )
27 ORDER BY s.student_id
28 LIMIT 5;

```

[실행결과]

```

ice2/problem18.sql
학번 | 이름 | 전공 | GPA
-----|-----|-----|-----
11 | Student_11 | EE | 3.10
12 | Student_12 | ME | 3.20
13 | Student_13 | CE | 3.30
14 | Student_14 | BIO | 3.40
15 | Student_15 | CS | 3.50
(5 rows)

```

19번

[SQL]

```

sql > practice2 > problem19.sql
1  -- 문제 19
2  -- HR 학과 학생 일부와의 비교 데모
3  -- HR 학과 최저 GPA 이상의 GPA를 가진 학생
4  SELECT
5      s.student_id as "학번",
6      s.name as "이름",
7      s.major as "전공",
8      s.gpa as "GPA"
9  FROM student s
10 WHERE s.gpa > ANY (
11     SELECT gpa
12     FROM student
13     WHERE major = 'HR'
14 )
15 ORDER BY s.student_id
16 LIMIT 5;
17
18 -- HR 학과 최고 GPA 이상의 GPA를 가진 학생
19 SELECT
20     s.student_id as "학번",
21     s.name as "이름",
22     s.major as "전공",
23     s.gpa as "GPA"
24 FROM student s
25 WHERE s.gpa >= ALL (
26     SELECT s2.gpa
27     FROM student s2
28     WHERE s2.major = 'HR'
29 )
30 ORDER BY s.student_id
31 LIMIT 5;

```

[실행결과]

```

ice2/problem19.sql
학번 | 이름 | 전공 | GPA
-----|-----|-----|-----
1 | Student_1 | EE | 2.10
2 | Student_2 | ME | 2.20
3 | Student_3 | CE | 2.30
4 | Student_4 | BIO | 2.40
5 | Student_5 | CS | 2.50
(5 rows)

학번 | 이름 | 전공 | GPA
-----|-----|-----|-----
29 | Student_29 | BIO | 4.90
59 | Student_59 | BIO | 4.90
89 | Student_89 | BIO | 4.90
119 | Student_119 | BIO | 4.90
149 | Student_149 | BIO | 4.90
(5 rows)

```

20번

[SQL]

```
sql > practice2 > problem20.sql
1  -- 문제 20
2  -- CS 학과 학생 또는 DB 과목을 수강한 학생 목록
3  SELECT
4      s.student_id,
5      s.name,
6      s.major,
7      s.gpa
8  FROM student s
9  WHERE s.major = 'CS' -- CS 학과
10 OR EXISTS ( -- DB 과목 수강
11     SELECT 1
12     FROM enroll e
13     WHERE e.course = 'DB'
14 )
15 ORDER BY s.student_id
16 LIMIT 5;
```

[실행결과]

```
ice2/problem20.sql
student_id | name      | major | gpa
-----+-----+-----+-----
1 | Student_1 | EE    | 2.10
2 | Student_2 | ME    | 2.20
3 | Student_3 | CE    | 2.30
4 | Student_4 | BIO   | 2.40
5 | Student_5 | CS    | 2.50
(5 rows)
```

21번

[SQL]


```

sql > practice2 > problem21.sql
1  -- 문제 21
2  -- student 테이블에는 학과(major)와 GPA가 있습니다.
3  -- 학과별·GPA 구간별 인원을 집계하되,
4  -- 소계(소계행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력
5  -- 1. GPA를 구간 (3.0 미만 / 3.0~3.5 / 3.5 초과)으로 분류하는 파생 컬럼을 추가
6  -- 2. GROUP BY ROLLUP(major, gpa_tier)로 학과별, 전체 소계를 동시에 조회
7  -- 3. GROUPING(major) 함수로 소계 행에 '전체' 라벨을 붙일 것
8  -- 4. 결과를 major, gpa_tier 순으로 정렬하되 소계 행은 하단에 표시
9  SELECT
10     CASE GROUPING(major) WHEN 1 THEN '전체' -- NULL -> 전체
11     | ELSE major
12     END as major,
13     CASE GROUPING(gpa_tier) WHEN 1 THEN '소계' -- NULL -> 소계
14     | ELSE gpa_tier
15     END as gpa_tier,
16     COUNT(*) AS cnt,
17     AVG(gpa)::NUMERIC(4, 2) AS avg_gpa
18 FROM (
19     -- GPA 구간별 파생컬럼
20     SELECT
21         *,
22         CASE WHEN gpa < 3.0 THEN 'C'
23         | WHEN gpa <= 3.5 THEN 'B'
24         | ELSE 'A'
25         END as gpa_tier
26     FROM student
27 ) t
28 GROUP BY ROLLUP(major, gpa_tier)
29 -- 일반 데이터 -> 학과별 소계 -> 전체 총합
30 ORDER BY GROUPING(major), major, GROUPING(gpa_tier), gpa_tier
31 LIMIT 5;

```

[실행결과]

```
ice2/problem21.sql
```

major	gpa_tier	cnt	avg_gpa
BIO	A	97	4.39
BIO	B	33	3.40
BIO	C	66	2.65
BIO	소계	196	3.64
CE	A	97	4.29

(5 rows)

22번

[SQL]

```

sql > practice2 > problem22.sql
8   WITH RECURSIVE org_tree as (
9       SELECT
10          manager_id,
11          0 as depth,
12          name::TEXT as path
13      FROM emp
14      WHERE manager_id IS NULL
15      UNION ALL
16
17      -- 재귀 단계: 하위 직원 추가
18      SELECT
19          e.emp_id,
20          e.name,
21          e.manager_id,
22          ot.depth + 1,
23          ot.path || ' > ' || e.name
24      FROM emp e
25      JOIN org_tree ot ON ot.emp_id = e.manager_id
26      WHERE ot.depth < 10 -- 무한 재귀 방지 !!! 항상 필요 !!!
27  ),
28  direct_count AS (
29      -- 매니저별 직속 부하 직원 수 집계
30      SELECT
31          manager_id,
32          COUNT(*) AS cnt
33      FROM emp
34      WHERE manager_id IS NOT NULL
35      GROUP BY manager_id
36  )
37  SELECT
38      depth,
39      REPEAT(' ', depth - 1) || name AS indented_name,
40      path,
41      dc.cnt as direct_reports
42  FROM org_tree
43  LEFT JOIN direct_count dc ON dc.manager_id = org_tree.emp_id
44  ORDER BY path;

```

[실행결과]

```
ice2/problem22.sql
```

depth	indented_name	path	direct_reports
0	CEO	CEO	40
1	Dev_102	CEO > Dev_102	
1	Dev_112	CEO > Dev_112	
1	Dev_12	CEO > Dev_12	
1	Dev_122	CEO > Dev_122	
1	Dev_132	CEO > Dev_132	
1	Dev_142	CEO > Dev_142	
1	Dev_152	CEO > Dev_152	
1	Dev_162	CEO > Dev_162	
1	Dev_172	CEO > Dev_172	
1	Dev_182	CEO > Dev_182	
1	Dev_192	CEO > Dev_192	
1	Dev_202	CEO > Dev_202	
1	Dev_212	CEO > Dev_212	
1	Dev_22	CEO > Dev_22	
1	Dev_222	CEO > Dev_222	
1	Dev_232	CEO > Dev_232	
1	Dev_242	CEO > Dev_242	
1	Dev_252	CEO > Dev_252	
1	Dev_262	CEO > Dev_262	
1	Dev_272	CEO > Dev_272	
1	Dev_282	CEO > Dev_282	
1	Dev_292	CEO > Dev_292	
1	Dev_302	CEO > Dev_302	
1	Dev_32	CEO > Dev_32	
1	Dev_42	CEO > Dev_42	
1	Dev_52	CEO > Dev_52	
1	Dev_62	CEO > Dev_62	
1	Dev_72	CEO > Dev_72	
1	Dev_82	CEO > Dev_82	
1	Dev_92	CEO > Dev_92	
1	Mgr_10	CEO > Mgr_10	30
2	Dev_101	CEO > Mgr_10 > Dev_101	
2	Dev_111	CEO > Mgr_10 > Dev_111	
2	Dev_121	CEO > Mgr_10 > Dev_121	
2	Dev_131	CEO > Mgr_10 > Dev_131	
2	Dev_141	CEO > Mgr_10 > Dev_141	
2	Dev_151	CEO > Mgr_10 > Dev_151	
2	Dev_161	CEO > Mgr_10 > Dev_161	

23번

[SQL]

```

sql > practice2 > problem23.sql
1  -- 문제 23
2  -- 각 학과(major)별로 GPA가 높은 순서로 순위를 매기고,
3  -- 상위 3명씩만 추출하는 쿼리를 Window Function으로 작성
4  -- (서브쿼리 방식과 CTE 방식 모두 작성)
5  -- 1. 학과 내 순위를 계산
6  -- ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC)
7  -- 2. GPA가 동일한 경우 student_id 오름차순을 2차 기준으로 사용
8  -- 3. RANK()와 DENSE_RANK()를 함께 계산해 동점 처리 방식 차이를 비교
9  -- 4. 결과에 전체 학과별 학생 수(total_in_major)도 COUNT() OVER(PARTITION BY major)로 추가
10 -- CTE
11 with ranked as (
12     SELECT
13         student_id,
14         name,
15         major,
16         gpa,
17         ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) as rn,
18         RANK() OVER (PARTITION BY major ORDER BY gpa DESC) as rnk,
19         DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drk,
20         COUNT(*) OVER (PARTITION BY major) AS total_in_major
21     FROM student
22 )
23 SELECT
24     *
25 FROM ranked
26 WHERE rn <= 3
27 ORDER BY major, rn;
28
29 -- 서브쿼리
30 SELECT
31     *
32 FROM (
33     SELECT
34         student_id,
35         name,
36         major,
37         gpa,
38         ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) as rn,
39         RANK() OVER (PARTITION BY major ORDER BY gpa DESC) as rnk,
40         DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drk,
41         COUNT(*) OVER (PARTITION BY major) AS total_in_major
42     FROM student
43 ) ranked
44 WHERE rn <= 3
45 ORDER BY major, rn;

```

[실행결과]

3.sql

student_id	name	major	gpa	rn	rnk	drk	total_in_major
29	Student_29	BIO	4.90	1	1	1	196
59	Student_59	BIO	4.90	2	1	1	196
89	Student_89	BIO	4.90	3	1	1	196
28	Student_28	CE	4.80	1	1	1	196
58	Student_58	CE	4.80	2	1	1	196
88	Student_88	CE	4.80	3	1	1	196
25	Student_25	CS	4.50	1	1	1	196
55	Student_55	CS	4.50	2	1	1	196
85	Student_85	CS	4.50	3	1	1	196
26	Student_26	EE	4.60	1	1	1	196
56	Student_56	EE	4.60	2	1	1	196
86	Student_86	EE	4.60	3	1	1	196
989	Student_989	HR	4.90	1	1	1	20
988	Student_988	HR	4.80	2	2	2	20
987	Student_987	HR	4.70	3	3	3	20
27	Student_27	ME	4.70	1	1	1	196
57	Student_57	ME	4.70	2	1	1	196
87	Student_87	ME	4.70	3	1	1	196

(18 rows)

student_id	name	major	gpa	rn	rnk	drk	total_in_major
29	Student_29	BIO	4.90	1	1	1	196
59	Student_59	BIO	4.90	2	1	1	196
89	Student_89	BIO	4.90	3	1	1	196
28	Student_28	CE	4.80	1	1	1	196
58	Student_58	CE	4.80	2	1	1	196
88	Student_88	CE	4.80	3	1	1	196
25	Student_25	CS	4.50	1	1	1	196
55	Student_55	CS	4.50	2	1	1	196
85	Student_85	CS	4.50	3	1	1	196
26	Student_26	EE	4.60	1	1	1	196
56	Student_56	EE	4.60	2	1	1	196
86	Student_86	EE	4.60	3	1	1	196
989	Student_989	HR	4.90	1	1	1	20
988	Student_988	HR	4.80	2	2	2	20
987	Student_987	HR	4.70	3	3	3	20
27	Student_27	ME	4.70	1	1	1	196
57	Student_57	ME	4.70	2	1	1	196
87	Student_87	ME	4.70	3	1	1	196

(18 rows)

24번

[SQL]

```

sql > practice2 > problem24.sql
1  -- 문제 24
2  -- enroll 테이블을 student_id 기준으로 정렬하고,
3  -- 각 학생의 이전 수강 과목 대비 성적 변화를 LAG()로 계산
4  -- (성적은 A=4, B=3, C=2, D=1 점수로 변환)
5  -- 1. grade를 숫자 점수(A=4, B=3, C=2, D=1)로 변환하는 CASE 식을 작성
6  -- 2. LAG(score) OVER (PARTITION BY student_id ORDER BY course)로 이전 과목 점수를 가져올 것
7  -- 3. 현재 점수 - 이전 점수를 diff 컬럼으로 추가하고, 상승/유지/하락을 텍스트로 표시
8  -- 4. 학생별 최고점과 최저점의 차이(score_range)를 Window Function으로 계산
9  with scored as (
10     SELECT
11         student_id,
12         course,
13         grade,
14         CASE grade WHEN 'A' THEN 4
15              WHEN 'B' THEN 3
16              WHEN 'C' THEN 2
17              ELSE 1
18         END as score
19     FROM enroll
20 ),
21 lagged as (
22     SELECT
23         *,
24         LAG(score) OVER (PARTITION BY student_id ORDER BY course) as prev_score,
25         MAX(score) OVER (PARTITION BY student_id) - MIN(score) OVER (PARTITION BY student_id) as score_range
26     FROM scored
27 )
28 SELECT
29     *,
30     score - prev_score as diff,
31     CASE WHEN score > prev_score THEN '상승'
32          WHEN score = prev_score THEN '유지'
33          ELSE '하락'
34     END as trend
35 FROM lagged;

```

[실행결과]

student_id	course	grade	score	prev_score	score_range	diff	trend
1	Course_3	C	2		3		하락
1	Course_4	D	1	2	3	-1	하락
1	Course_5	A	4	1	3	3	상승
2	Course_4	D	1		3		하락
2	Course_5	A	4	1	3	3	상승
3	Course_5	A	4		2		하락
3	Course_6	B	3	4	2	-1	하락
3	Course_7	C	2	3	2	-1	하락
4	Course_6	B	3		1		하락
4	Course_7	C	2	3	1	-1	하락
5	Course_7	C	2		3		하락
5	Course_8	D	1	2	3	-1	하락
5	Course_9	A	4	1	3	3	상승
6	Course_8	D	1		3		하락
6	Course_9	A	4	1	3	3	상승
7	Course_10	B	3		2		하락
7	Course_11	C	2	3	2	-1	하락
7	Course_9	A	4	2	2	2	상승

25번

[SQL]

```
sql > practice2 > problem25.sql
4  -- 2. AVG(amount) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)로 3개 이동평균 계산
5  -- 3. customer_id별로 PARTITION을 나눠 고객별 누적 구매금액도 함께 계산
6  -- 4. 누적합이 전체 합의 50%를 초과하는 첫 번째 order_id를 찾는 쿼리를 작성
7  WITH total_orders as (
8      SELECT SUM(amount) as total_sum
9      FROM orders
10 ),
11 order_stats AS (
12     SELECT
13         order_id,
14         customer_id,
15         amount,
16         SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) as cum_total,
17         AVG(amount) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) as moving_avg_3,
18         SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) as cust_cum
19     FROM orders
20 )
21 SELECT
22     s.order_id,
23     s.customer_id,
24     s.amount,
25     s.cum_total,
26     s.moving_avg_3,
27     s.cust_cum
28 FROM order_stats s
29 CROSS JOIN total_orders t
30 WHERE s.cum_total >= t.total_sum * 0.5
31 ORDER BY s.order_id
32 LIMIT 1;
```

[실행결과]

```
heejoo@munhuijuui-MacBookPro skala-postgre % psql -U skala_user -d skala_db -f sql/practice2/problem25.sql
order_id | customer_id | amount | cum_total | moving_avg_3 | cust_cum
-----+-----+-----+-----+-----+-----
1503 | 4 | 4544.03 | 3686585.56 | 4531.0200000000000000 | 9176.12
(1 row)
```